

Compte rendu TP1

Initiation à MATLAB et fonction de transfert

Travail par :

Ahmed CHAABANE

Firas KAHLAOUI

Seifeddine BEN FRADJ

1.Introduction

Dans le cadre de ce TP, nous allons utiliser Matlab pour faire la modélisation d'un moteur MCC un Maxon F 2260, numéro 885 pour déterminer la fonction de transfert et des différentes caractéristiques.

2.Partie pratique

Dans cette partie on va détailler les étapes suivies pour faire l'étude de système en Matlab et les résultats obtenus.

Question 1 :

On a créé un nouveau script nommée 'script_tp1.m' pour écrire tout le code.

Question 2 :

Dans cette partie on a initialisé les caractéristiques du système qu'on va étudier, dans ce cas un MCC un Maxon F 2260, numéro 885, en utilisant le script suivant, le résultat est d'obtenir les nouvelles valeurs dans 'Workspace'

```
R = 1.44;  
L = 5.610 * 10^-4;  
f = 7.2 * 10^-5;  
J = 1.29 * 10^-4;  
Kem = 0.1;
```

Workspace			
Name	Value	Size	Class
f	7.2000e-05	1×1	double
J	1.2900e-04	1×1	double
Kem	0.1000	1×1	double
L	5.6100e-04	1×1	double
R	1.4400	1×1	double

Figure 1: Affichage des valeurs des caractéristiques dans Workspace

Question 3:

Pour cette partie on a calculé les valeurs de gain K et les constantes de temps électrique τ_{el} et électromécanique τ_{em} du MCC comme la suite :

```
K = Kem / (R * f + Kem^2)
Tem = (R*J) / (R*f + Kem ^ 2)
Tel = L / R
```

On a obtenu les résultats suivants sur la ligne de commande:

```
K =
    9.8974

Tem =
    0.0184

Tel =
    3.8958e-04
```

Question 4:

Pour vérifier les valeurs des pôles il suffit de déterminer la fonction de transfert G puis calculer les pôles et les comparer avec les valeurs de $1/\tau_{em}$ et $1/\tau_{el}$

```
G = tf([K], [Tel*Tem Tem+Tel 1])
pole(G)
1/Tel
1/Tem
```

On obtient les résultats suivants:

```
G =

          9.897
-----
7.163e-06 s^2 + 0.01877 s + 1

Continuous-time transfer function.
Model Properties
```

```
ans =  
    1.0e+03 *  
  
    -2.5668  
    -0.0544  
  
ans =  
    2.5668e+03  
  
ans =  
    54.3910
```

On remarque que les résultats sont identiques, d'où notre travail est correct.

Question 5:

On a utilisé la commande 'help figure' et 'help subplot' et on a obtenu les résultats suivants:

```
>> help figure  
figure - Create figure window  
This MATLAB function creates a new figure window using default property  
values.  
  
Syntax  
    figure  
    figure(Name,Value)  
    f = figure(___)  
  
    figure(f)  
    figure(n)  
  
Input Arguments  
    f - Target figure  
        Figure object  
    n - Target figure number  
        scalar integer value  
  
Name-Value Arguments  
    Name - Name  
        '' (default) | character vector | string scalar  
    Color - Background color  
        RGB triplet | hexadecimal color code | 'r' | 'g' | 'b' | ...  
    Position - Location and size of figure, excluding borders, figure tools, and ti-  
tle bar  
        [left bottom width height]  
    Units - Units of measurement  
        'pixels' (default) | 'normalized' | 'inches' | 'centimeters' |  
        'points' | 'characters'  
  
Examples  
    Change Figure Size  
    Specify Figure Title  
    Working with Multiple Figures Simultaneously
```

```
>> help subplot  
subplot - Create axes in tiled positions  
This MATLAB function divides the current figure into an m-by-n grid and
```

creates axes in the position specified by p.

Syntax

```
subplot(m,n,p)
subplot(m,n,p,'replace')
subplot(m,n,p,'align')
subplot(m,n,p,ax)
```

```
subplot('Position',pos)
```

```
subplot(___,Name,Value)
ax = subplot(___)
```

```
subplot(ax)
```

Input Arguments

[m](#) - Number of grid rows
1 (default) | positive integer

[n](#) - Number of grid columns
1 (default) | positive integer

[p](#) - Grid position for new axes
scalar | vector

[pos](#) - Custom position for new axes
four-element vector

[ax](#) - Existing axes to make current or convert to subplot
Axes object | PolarAxes object | GeographicAxes object |
graphics object

Examples

[Upper and Lower Subplots](#)
[Quadrant of Subplots](#)
[Subplots with Different Sizes](#)
[Replace Subplot with Empty Axes](#)
[Subplots at Custom Positions](#)
[Create Subplots with Polar Axes](#)
[Modify Axes Properties After Creation](#)
[Make Subplot the Current Axes](#)
[Convert Existing Axes to Subplot](#)
[Convert Axes in Separate Figures to Subplots](#)

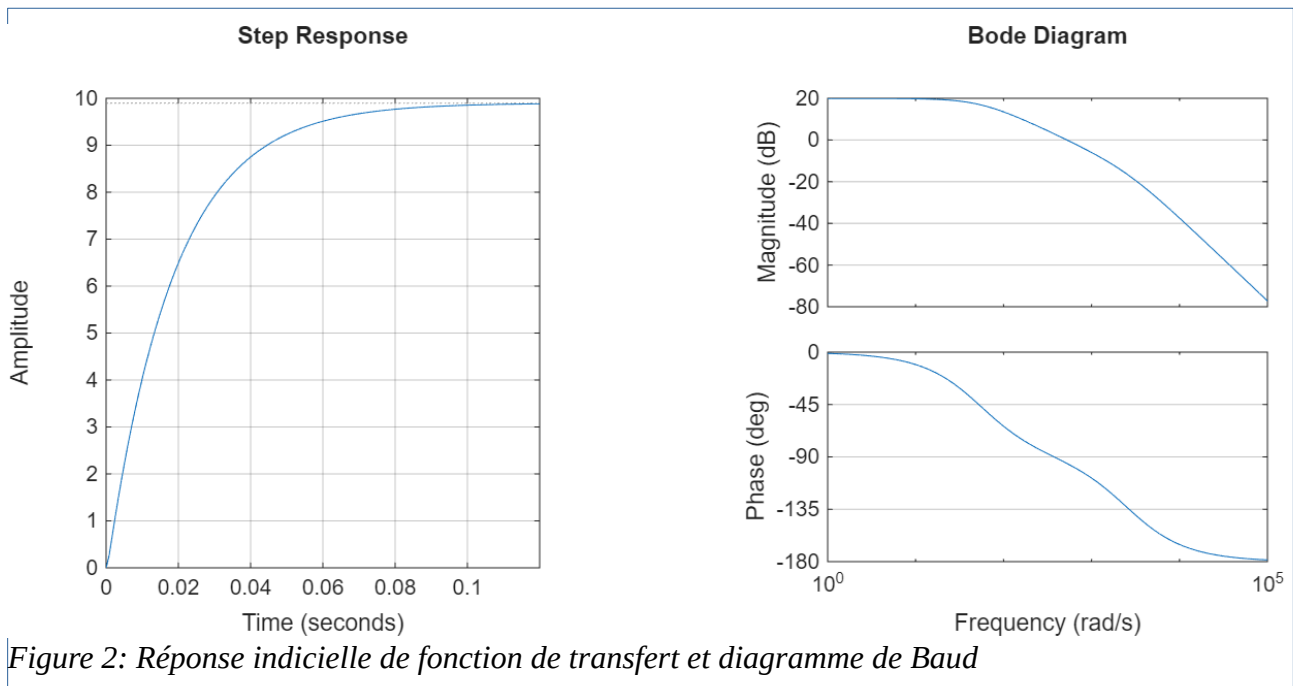
Question 6:

On utilise les commandes suivantes pour créer les figures de la réponse indicielle et le diagramme de Baud:

```
figure(1)
subplot(121)
step(G)
grid on;

subplot(122)
bode(G)
grid on;
```

On obtient les résultats suivants:



Question 7:

On initialise K_p et on calcule K_ω en utilisant la formule suivante:

```
Kw = 10 / 3000
Kp = 10;
```

On retrouve que $K_\omega = 0.0033$

Question 8:

On initialise les fonctions de transfert en boucle ouvert et en boucle fermé en utilisant le script suivant:

```
Go = Kp * G * Kw
Gf = feedback(Kp * G, Kw)
```

On obtient les résultats suivants:

```
Go =
      0.3299
-----
7.163e-06 s^2 + 0.01877 s + 1
Continuous-time transfer function.
Model Properties

Gf =
      98.97
-----
7.163e-06 s^2 + 0.01877 s + 1.33
Continuous-time transfer function.
Model Properties
```

Question 9:

On utilise les commandes suivantes pour créer les figures de la réponse indicielle du système en boucle ouverte puis en boucle fermé:

```
figure(2)
subplot(121)
step(Go)

subplot(122)
step(Gf)
```

On obtient les résultats suivants :

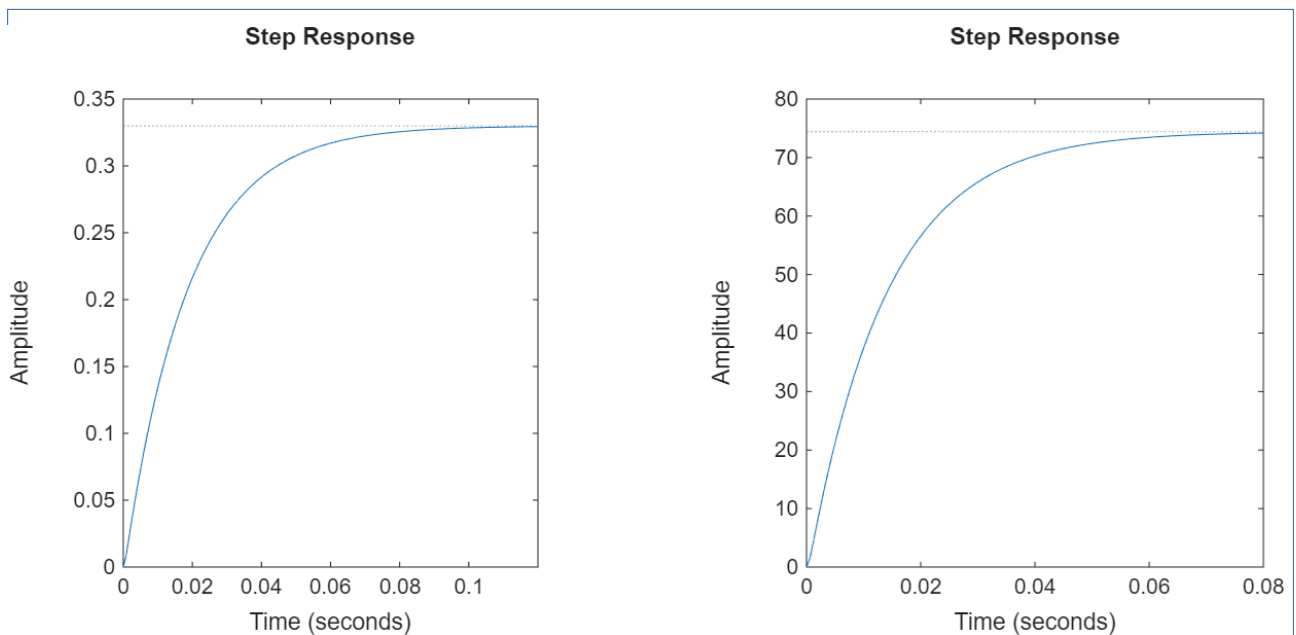


Figure 3: Réponse indicielle de fonction de transfert en boucle ouvert et en boucle fermé

Conclusion:

Dans ce TP, on a fait la modélisation d'un MCC un Maxon F 2260 avec ces caractéristiques physiques, après on a analysé sa réponse indicielle en fréquence et dans le domaine temporel, puis on a étudié la stabilité du système et on conclue que ce système est stable, et finalement on a démontré l'importance de la boucle fermée avec asservissement proportionnel et le correcteur proportionnel $K_p = 10$ et le capteur K_ω pour obtenir une meilleure régulation de la vitesse.